

פרוייקט רשתות תקשורת מחשבים

אוריה פרץ ואוריאן ראובן

חלק 1 - אריזה ולכידת המנות

אופן יצירת הקובץ

קובץ CSV הוכן באמצעות סקריפט בשפת Python שנכתב בסיוע מודל AI. באמצעות הקובץ ביצענו סימולציה של הודעות המדמות תעבורת HTTP בשכבת היישום כך שנוצר רצף הודעות המדמות שיחה בין דפדפן לשרת אינטרנט. כל שורה בקובץ CSV מייצגת הודעת יישום אחת כלומר בקשת HTTP או תגובת HTTP. הסימולציה בוצעה על ידי בחירה אקראית מתוך רשימה מוגדרת מראש של דפי אינטרנט, קבצים ושיטות בקשה מקובלות כמו GET וPOST. במקום להשתמש בזמנים סטטיים, דאגנו לרווח את ההודעות עם חותמות זמן עוקבות כדי לדמות השהיה טבעית של שיחה אמיתית. זה אפשר לנו לראות ב-Wireshark רצף אירועים ברור ומסודר, כשכל שורה בקובץ הופכת להודעה שנשלחת ברשת.

תהליך אריזת המנות

תהליך האריזה מתאר את האופן שבו נתוני שכבת היישום מועברים דרך שכבות מודל ה-TCP/IP עד לשידורם ברשת. בתחילת התהליך המידע עובר בשכבת האפליקציה ובה נוצרת הודעת ה-HTTP המהווה את תוכן המידע הגולמי. בשלב הבא, המידע עובר בשכבת התעבורה ובה הנתונים נארזים בתוך מקטע של פרוטוקול TCP, בשכבה זו מתווספים הפורטים של המקור והיעד, וכן שדות נוספים המשמשים לבקרת זרימה ואמינות התקשורת. לאחר מכן המידע עובר ומגיע לשכבת הרשת שבה המקטע נארז בתוך IP Packet, אשר כוללת את כתובות ה-IP של המקור והיעד לצורך ניתוב בין הרשתות השונות. בשלב האחרון המידע עובר לשכבת הקישור ובה המידע נארז במסגרת הכוללת כתובות MAC פיזיות. תהליך זה מאפשר את העברת המידע בתוך הרשת המקומית ועד הגעתו ליעד הסופי.

תהליך הלכידה

לכידת התעבורה בוצעה באמצעות תוכנת Wireshark, על ממשק הרשת Adapter for loopback, אשר מאפשר צפייה וניתוח של חבילות רשת בזמן אמת. הלכידה בוצעה במקביל להרצת מחברת jupyter, במטרה לבחון את הדינמיקה של תעבורת הנתונים שייצרנו. לצורך הניתוח הופעל מסנן תצוגה tcp.port==12345 על מנת לבודד את התעבורה הרלוונטית שייצרנו במחברת jupyter. במהלך הלכידה נשלחו חבילות TCP המכילות בקשות HTTP מדומות. בסימולציה זו, החבילות נשלחו לפורט שאינו מאזין על ידי שרת פעיל, ולכן מערכת ההפעלה החזירה חבילות מסוג RST המסומנות ב-Wireshark. חלק מתוכן ההודעות בשכבת היישום הופיע בלכידה ואפשר ניתוח של מבנה החבילות.

התעבורה שנלכדה ב-wireshark

התעבורה שנלכדה היא תעבורת Loopback פנימית של המחשב, ומטרת הלכידה הייתה לבחון כיצד הודעות היישום נארזות, מועברות ונראות בפועל ברשת, וכן לנתח את מבנה החבילות בכל אחת משכבות המודל. בקטע זה מוצגים צילומי מסך מתוך Wireshark, המדגימים את תהליך האריזה והעברת הנתונים, את מבנה החבילות שנלכדו, ואת תוכן שכבת היישום כפי שהוא מופיע בתוך חבילות ה-TCP. כל צילום מלווה בהסבר המתאר את המידע המוצג ואת משמעותו במסגרת תהליך התקשורת. במהלך הלכידה נלכדו 5344 חבילות, בקצב ממוצע של כ-74 חבילות לשנייה. גודל חבילה ממוצע עמד על 236 בתים, ונפח הנתונים הכולל שנלכד היה כ-1.26 MB.

ברשימת הפקטות שהתקבלו ניתן לראות את הדינמיקה בין תקשורת הלקוח לשרת.
 רצף הפקטות מציג מחזוריות של בקשות HTTP POST ובעקבותיהן תגובות HTTP 200 OK, דבר המעיד על עבודה תקינה של הסימולציה מול קובץ CSV שבו מוגדרים זוגות של בקשה ומענה.

POST /573e672b-3faa-4ad1-9048-0cccb363654c/ HTTP/1.1 797	HTTP/XML	1::	1::	12.500989
HTTP/1.1 200 2415	HTTP/XML	1::	1::	12.511426
POST /573e672b-3faa-4ad1-9048-0cccb363654c/ HTTP/1.1 777	HTTP/XML	127.0.0.1	127.0.0.1	12.531477
HTTP/1.1 200 2395	HTTP/XML	127.0.0.1	127.0.0.1	12.533649
POST /573e672b-3faa-4ad1-9048-0cccb363654c/ HTTP/1.1 777	HTTP/XML	127.0.0.1	127.0.0.1	12.558127
HTTP/1.1 200 2395	HTTP/XML	127.0.0.1	127.0.0.1	12.552060
POST /573e672b-3faa-4ad1-9048-0cccb363654c/ HTTP/1.1 797	HTTP/XML	1::	1::	12.568065
HTTP/1.1 200 2415	HTTP/XML	1::	1::	12.569432
GET /api/contents/Downloads?content=1&1766500278069 HTTP/1.1 1596	HTTP	1::	1::	20.172287
GET /api/kernels?1766500278093 HTTP/1.1 1575	HTTP	1::	1::	20.176248
GET /api/terminals?1766500278094 HTTP/1.1 1577	HTTP	1::	1::	20.177019
GET /api/sessions?1766500278095 HTTP/1.1 1576	HTTP	1::	1::	20.177595
HTTP/1.1 200 OK , JSON (application/json) 1222	HTTP/JSON	1::	1::	20.186452
HTTP/1.1 200 OK , JSON (application/json) 422	HTTP/JSON	1::	1::	20.189323
HTTP/1.1 200 OK , JSON (application/json) 2213	HTTP/JSON	1::	1::	20.192555
HTTP/1.1 200 OK , JSON (application/json) 16903	HTTP/JSON	1::	1::	20.292601
/api/kernels/2e330c1c-bdb4-49a0-96fa-baaf9df38de2/restapi?176650 1671	HTTP	1::	1::	24.020887
GET /static/favicons/favicon-busy-1.ico HTTP/1.1 1526	HTTP	1::	1::	24.058710
HTTP/1.1 304 Not Modified 371	HTTP	1::	1::	24.061171
HTTP/1.1 200 OK , JSON (application/json) 551	HTTP/JSON	1::	1::	25.758429
/api/kernels/2e330c1c-bdb4-49a0-96fa-baaf9df38de2/channels?sessio 1490	HTTP	1::	1::	25.773067
HTTP/1.1 101 Switching Protocols 316	HTTP	1::	1::	25.780686
GET /api/contents/Downloads?content=1&1766500283831 HTTP/1.1 1596	HTTP	1::	1::	25.877317
HTTP/1.1 200 OK , JSON (application/json) 16903	HTTP/JSON	1::	1::	26.093324
GET /static/favicons/favicon-busy-1.ico HTTP/1.1 1526	HTTP	1::	1::	28.498725
HTTP/1.1 304 Not Modified 371	HTTP	1::	1::	28.510402

ניתן לראות כי רוב התעבורה מתבצעת על גבי HTTP/XML המועבר בתוך IPv6 כתובת 1::.
 בנוסף ניתן לראות גם פקטות מסוג HTTP/JSON המעידות על חילופי מידע בין האפליקציות.

HTTP/1.1 200 OK , JSON (application/json) 2213	HTTP/JSON	1::	1::	20.192555
HTTP/1.1 200 OK , JSON (application/json) 16903	HTTP/JSON	1::	1::	20.292601
HTTP/1.1 200 OK , JSON (application/json) 551	HTTP/JSON	1::	1::	25.758429
HTTP/1.1 200 OK , JSON (application/json) 16903	HTTP/JSON	1::	1::	26.093324
HTTP/1.1 200 OK , JSON (application/json) 1222	HTTP/JSON	1::	1::	30.206107
HTTP/1.1 200 OK , JSON (application/json) 422	HTTP/JSON	1::	1::	30.211845
HTTP/1.1 200 OK , JSON (application/json) 2213	HTTP/JSON	1::	1::	30.214683
HTTP/1.1 200 OK , JSON (application/json) 16903	HTTP/JSON	1::	1::	37.073830
HTTP/1.1 200 OK , JSON (application/json) 912	HTTP/JSON	1::	1::	38.897770
HTTP/1.1 200 OK , JSON (application/json) 622	HTTP/JSON	1::	1::	38.902284
HTTP/1.1 200 OK , JSON (application/json) 1222	HTTP/JSON	1::	1::	40.229918
HTTP/1.1 200 OK , JSON (application/json) 422	HTTP/JSON	1::	1::	40.234579
HTTP/1.1 200 OK , JSON (application/json) 2213	HTTP/JSON	1::	1::	40.236990
HTTP/1.1 200 OK , JSON (application/json) 16903	HTTP/JSON	1::	1::	47.370684
HTTP/1.1 200 OK , JSON (application/json) 1222	HTTP/JSON	1::	1::	50.245292
HTTP/1.1 200 OK , JSON (application/json) 422	HTTP/JSON	1::	1::	50.248501
HTTP/1.1 200 OK , JSON (application/json) 2213	HTTP/JSON	1::	1::	50.252068
HTTP/1.1 200 OK , JSON (application/json) 16903	HTTP/JSON	1::	1::	57.739328
HTTP/1.1 200 OK , JSON (application/json) 1222	HTTP/JSON	1::	1::	60.264970
HTTP/1.1 200 OK , JSON (application/json) 422	HTTP/JSON	1::	1::	60.268534
HTTP/1.1 200 OK , JSON (application/json) 2213	HTTP/JSON	1::	1::	60.271538
HTTP/1.1 200 OK , JSON (application/json) 16903	HTTP/JSON	1::	1::	67.890433

ניתן להבחין שהתקשורת מתבצעת מול כתובות IP הקבועות שהינן 1::, ו- 127.0.0.1 דבר המעיד על כך שהסימולציה רצה באופן מקומי - כלומר הלקוח והשרת נמצאים על אותו מחשב ומדברים ביניהם ברשת פנימית.

					UDP	TCP · 20	IPv6 · 1	IPv4 · 1	Ethernet
Country	Rx Bytes	Rx Packets	Tx Bytes	Tx Packets	Percent Filtered	Total Packets	Bytes	Packets	Address
	kB 262	146	kB 262	146	10.72%	2,723	kB 524	292	1::

					UDP	TCP · 20	IPv6 · 1	IPv4 · 1	Ethernet
Country	Rx Bytes	Rx Packets	Tx Bytes	Tx Packets	Percent Filtered	Total Packets	Bytes	Packets	Address
	kB 13	8	kB 13	8	0.23%	6,998	kB 25	16	127.0.0.1

על מנת להדגים כיצד הודעות היישום שיצרנו נארוזות ועוברות ברשת, בחרנו לנתח את פקטה מספר 62 מהלכידה. פקטה זו מייצגת בקשת POST המופיעה בקובץ CSV שלנו בשורה 113.

15.551	POST /login.html HTTP/1.1	web_server	client_browser	HTTP	113
--------	---------------------------	------------	----------------	------	-----

בפקטה ניתן לראות כי הפרוטוקול בשכבת הרשת הינו IPv6. כתובות המקור והיעד זהות - ::1 ומהוות את כתובת Loopback המשמשת לתקשורת פנימית בין הלקוח לשרת המדומה על אותו המחשב כפי שהוגדר בסימולציה.

```
Frame 62: Packet, 797 bytes on wire (6376 bits), 797 bytes captured (6376 bits) on interface \Device\NPF_{Loopback, id 0 <
Null/Loopback <
Internet Protocol Version 6, Src: ::1, Dst: ::1 <
Version: 6 = ... 0110
Traffic Class: 0x00 (DSCP: CS0, ECN: Not-ECT) = ... 0000 0000 ... <
Flow Label: 0x64e13 = 0011 0001 1110 0100 0110 ...
Payload Length: 753
Next Header: TCP (6)
Hop Limit: 128
Source Address: ::1 <
Destination Address: ::1 <
[Stream index: 0]
Transmission Control Protocol, Src Port: 59215, Dst Port: 5357, Seq: 219, Ack: 1, Len: 733 <
[Reassembled TCP Segments (951 bytes): #60(218), #62(733) 2] <
Hypertext Transfer Protocol <
eXtensible Markup Language <
xml?> <
soap:Envelope> <
```

בשכבת התעבורה נעשה שימוש בפרוטוקול TCP אשר תומך בהעברת נתונים אמינה ורציפה, ונדרש על מנת להבטיח שבקשת ה-POST מקובץ ה-CSV תעבור את כל הדרך מהלקוח לשרת ללא איבוד מידע. ניתן לראות כי נוספו Headers המכילים את פורט המקור (59215) ואת פורט היעד (5357), וכן את מספר הרצף האחראי על סידור הנתונים, בנוסף ניתן לראות שהדגל Push מופעל מה שמורה למערכת להעביר את המידע לשכבת היישום.

```
Transmission Control Protocol, Src Port: 59215, Dst Port: 5357, Seq: 219, Ack: 1, Len: 733 <
Source Port: 59215
Destination Port: 5357
[Stream index: 5]
[Stream Packet Number: 6]
[Conversation completeness: Complete, WITH_DATA (31)] <
[TCP Segment Len: 733]
Sequence Number: 219 (relative sequence number)
Sequence Number (raw): 1158520873
[Next Sequence Number: 952 (relative sequence number)]
Acknowledgment Number: 1 (relative ack number)
Acknowledgment number (raw): 263244643
Header Length: 20 bytes (5) = ... 0101
Flags: 0x018 (PSH, ACK) <
Window: 255
[Calculated window size: 65280]
[Window size scaling factor: 256]
Checksum: 0xc646 [unverified]
[Checksum Status: Unverified]
Urgent Pointer: 0
```

בשכבת היישום נעשה שימוש בפרוטוקול HTTP, בשכבה זו מזהה בקשת HTTP POST אשר מקבילה לפקודה בקובץ הCSV.

```
Hypertext Transfer Protocol
POST /573e672b-3faa-4ad1-9048-0cccb363654c/ HTTP/1.1
Request Method: POST
/Request URI: /573e672b-3faa-4ad1-9048-0cccb363654c
Request Version: HTTP/1.1
Cache-Control: no-cache
Connection: Keep-Alive
Pragma: no-cache
Content-Type: application/soap+xml
User-Agent: WSDAPI
Content-Length: 733
Host: [::1]:5357

[Response in frame: 66]
[/Full request URI: http://[::1]:5357/573e672b-3faa-4ad1-9048-0cccb363654c]
File Data: 733 bytes
```

ניתן לראות בשכבה זו את המידע הגולמי שנארז בתוך כל שכבות המודל האחרות ומכיל את הפרמטרים שנשלחו מהלקוח לשרת, ומופיע בתצורת XML, כאשר תוכן ההודעה מועבר בפורמט XML בתוך הPayload. ניתן לראות את מבנה ההודעה הכולל כותרות יישום וגוף ההודעה המכיל את הנתונים המקוריים שנשלחו.

```
eXtensible Markup Language
xml?>
"version="1.0
"encoding="utf-8
<?
soap:Envelope>
"xmlns:soap="http://www.w3.org/2003/05/soap-envelope
"xmlns:wsa="http://schemas.xmlsoap.org/ws/2004/08/addressing
<"xmlns:lms="http://schemas.microsoft.com/windows/lms/2007/08
<soap:Header>
<wsa:To>
<wsa:Action>
<wsa:MessageID>
<wsa:ReplyTo>
<wsa:From>
</lms:LargeMetadataSupport>
<soap:Header/>
</soap:Body>
<soap:Envelope/>
```

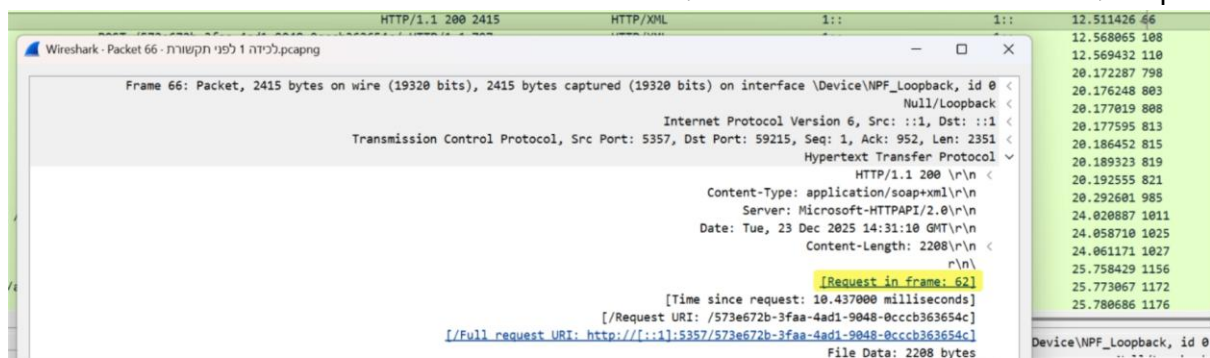
בצילום זה ניתן לראות את רצף התקשורת המלא בשכבת היישום, כולל הנתונים הגולמיים כפי שהם עוברים בין הלקוח לשרת. ניתן לראות את הודעת ה-POST המייצגת את הבקשה שנשלחה מהלקוח, כולל כותרות ה-HTTP וה-Payload המלא בפורמט XML, המכיל את תוכן ההודעה המקורי כפי שהוגדר בקובץ ה-CSV (שורה 113). בנוסף מופיעה הודעת התגובה מהשרת מסוג HTTP/1.1 200 OK, המעידה על כך שהשרת קיבל ועיבד את הבקשה בהצלחה, בהתאם לשורה 114 בקובץ ה-CSV.

```
POST /573e672b-3faa-4ad1-9048-0cccb363654c/ HTTP/1.1
Cache-Control: no-cache
Connection: Keep-Alive
Pragma: no-cache
Content-Type: application/soap+xml
User-Agent: WSDAPI
Content-Length: 733
Host: [::1]:5357
```

```
<?xml version="1.0" encoding="utf-8"?><soap:Envelope xmlns:soap="http://www.w3.org/2003/05/soap-envelope" xmlns:wsa="http://schemas.xmlsoap.org/ws/2004/08/addressing" xmlns:wsx="http://schemas.xmlsoap.org/ws/2004/09/mex" xmlns:wsdp="http://schemas.xmlsoap.org/ws/2006/02/devprof" xmlns:un0="http://schemas.microsoft.com/windows/pnpx/2005/10" xmlns:pub="http://schemas.microsoft.com/windows/pub/2005/07"><soap:Header><wsa:To>http://schemas.xmlsoap.org/ws/2004/08/addressing/role/anonymous</wsa:To><wsa:Action>http://schemas.xmlsoap.org/ws/2004/09/transfer/Get</wsa:Action><wsa:MessageID>urn:uuid:ab59a3d0-651d-4b55-a70e-78a7fd735ba6</wsa:MessageID><wsa:ReplyTo><wsa:Address>http://schemas.xmlsoap.org/ws/2004/08/addressing/role/anonymous</wsa:Address></wsa:ReplyTo><wsa:From><wsa:Address>urn:uuid:bba1824f-f5cd-4172-b0de-56b11d13cc49</wsa:Address></wsa:From></wsa:Header><soap:Body></soap:Body></soap:Envelope>
HTTP/1.1 200
Content-Type: application/soap+xml
Server: Microsoft-HTTPAPI/2.0
Date: Tue, 23 Dec 2025 14:31:10 GMT
Content-Length: 2208
```

```
<?xml version="1.0" encoding="utf-8"?><soap:Envelope xmlns:soap="http://www.w3.org/2003/05/soap-envelope" xmlns:wsa="http://schemas.xmlsoap.org/ws/2004/08/addressing" xmlns:wsx="http://schemas.xmlsoap.org/ws/2004/09/mex" xmlns:wsdp="http://schemas.xmlsoap.org/ws/2006/02/devprof" xmlns:un0="http://schemas.microsoft.com/windows/pnpx/2005/10" xmlns:pub="http://schemas.microsoft.com/windows/pub/2005/07"><soap:Header><wsa:To>http://schemas.xmlsoap.org/ws/2004/08/addressing/role/anonymous</wsa:To><wsa:Action>http://schemas.xmlsoap.org/ws/2004/09/transfer/GetResponse</wsa:Action><wsa:MessageID>urn:uuid:56364996-1fd5-4f0a-bad8-2c1487d77910</wsa:MessageID><wsa:RelatesTo>urn:uuid:ab59a3d0-651d-4b55-a70e-78a7fd735ba6</wsa:RelatesTo></soap:Header><soap:Body><wsx:MetadataSection Dialect="http://schemas.xmlsoap.org/ws/2006/02/devprof/ThisDevice"><wsdp:ThisDevice><wsdp:FriendlyName>Microsoft Publication Service Device Host</wsdp:FriendlyName><wsdp:FirmwareVersion>1.0</wsdp:FirmwareVersion><wsdp:SerialNumber>20050718</wsdp:SerialNumber></wsdp:ThisDevice></wsx:MetadataSection><wsx:MetadataSection Dialect="http://schemas.xmlsoap.org/ws/2006/02/devprof/ThisModel"><wsdp:ThisModel><wsdp:Manufacturer>Microsoft Corporation</wsdp:Manufacturer><wsdp:ManufacturerUrl>http://www.microsoft.com</wsdp:ManufacturerUrl><wsdp:ModelName>Microsoft Publication Service</wsdp:ModelName><wsdp:ModelNumber>1</wsdp:ModelNumber><wsdp:ModelUrl>http://www.microsoft.com</wsdp:ModelUrl><wsdp:PresentationUrl>http://www.microsoft.com</wsdp:PresentationUrl><un0:DeviceCategory>Computers</un0:DeviceCategory></wsdp:ThisModel></wsx:MetadataSection><wsx:MetadataSection Dialect="http://schemas.xmlsoap.org/ws/2006/02/devprof/Relationship"><wsdp:Relationship Type="http://schemas.xmlsoap.org/ws/2006/02/devprof/host"><wsdp:Host><wsa:EndpointReference><wsa:Address>urn:uuid:573e672b-3faa-4ad1-9048-0cccb363654c</wsa:Address></wsa:EndpointReference></wsdp:Types><wsdp:ServiceId>urn:uuid:573e672b-3faa-4ad1-9048-0cccb363654c</wsdp:ServiceId><pub:Computer>DESKTOP-2L1155P</pub:Computer></wsdp:Host></wsdp:Relationship></wsx:MetadataSection></wsx:MetadataSection></soap:Body></soap:Envelope>
```

באמצעות ניתוח פקטה 62 ניתן לראות באופן ויזואלי את תהליך אריזת המידע: הודעת היישום HTTP POST נארזת בתוך מעטפת TCP, ולאחר מכן מוכנסת לתוך מעטפת ה-IP עד ליצירת ה-Frame מלא שנשלח בשרת. תהליך זה מדגים כיצד כל שכבה מוסיפה מידע על מנת שהמסר המקורי מהקובץ יגיע ליעדו בצורה תקינה. בפקטה 66 ניתן לראות את תגובת השרת מסוג HTTP 200 OK המאשרת כי המידע שנשלח ב-Payload התקבל ועובד בהצלחה בשכבת היישום של היעד.



חלק 2 - יצירת יישום וניתוח תעבורה

הסבר כללי על המערכת ומבנה הקוד

המערכת היא אפליקציית צ'אט מבוססת שרת-לקוח, המשתמשת בפרוטוקול TCP לאמינות התעבורה. המערכת נכתבה בשפת Python תוך שימוש בספריית socket לתקשורת וספריית threading לטיפול במספר לקוחות במקביל. השרת מנהל את רשימת המחברים ומצליב הודעות בין זוגות משתמשים.

הוראות התקנה והרצה

- (1) לוודא שמוותקן Python 3
- (2) להריץ את השרת: Python server.py (אנחנו בחרנו להריץ בתוכנת VISUAL STUDIO CODE)
- (3) להריץ את הלקוח: Python client.py (ניתן להריץ מספר פעמים בטרמינלים נפרדים)
- (4) הכנס את שם המשתמש
- (5) על מנת לשוחח עם לקוח ספציפי הקלד '/chat <name>' כדי להתחיל שיחה

דוגמאות קלט ופלט

טרמינל שרת:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python + v [ ] [ ] ... | [ ] [ ] X
```

```
PS C:\Users\oriya\PyProject> & C:/Users/oriya/AppData/Local/Python/pythoncore-3.14-64/python.exe c:/Users/oriya/PyProject/SERVER.PY
Server is running on 0.0.0.0:5555...
Tommy connected!
Miley connected!
Fibi connected!
Mike connected!
Will connected!
```

טרמינלים לקוח 1 ולקוח 2:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
```

```
PS C:\Users\orian\Desktop\project1> & C:\Users\orian\AppData\Local\Programs\Python\Python314\python.exe c:/Users/orian/Desktop/project1/client1.py
Choose your nickname: Tommy
Welcome Tommy! Type '/chat <name>' to start.

/chat Miley
Connected to Miley!

Hello Miley,how are you?
Miley: good
```

```
PS C:\Users\orian\Desktop\project1> python client2.py
Choose your nickname: Miley
Welcome Miley! Type '/chat <name>' to start.

Tommy wants to chat with you!

Tommy: Hello Miley,how are you?

good
```


טרמינלים לקוח 3, לקוח 4 ולקוח 5 (שנכנס במקום לקוח 4 לצ'אט עם לקוח 3):

```
PS C:\Users\orian\Desktop\project1> python client4.py
Choose your nickname: Mike
Welcome Mike! Type '/chat <name>' to start.
```

```
/chat Fibi
Connected to Fibi!
```

```
Hello Fibi
Fibi: Hi Mike
```

```
█
```

```
PS C:\Users\orian\Desktop\project1> python client3.py
Choose your nickname: Fibi
Welcome Fibi! Type '/chat <name>' to start.
```

```
Mike wants to chat with you!
```

```
Mike: Hello Fibi
```

```
Hi Mike
Will wants to chat with you!
```

```
Will: hello Fibi
```

```
Hey
```

```
█
```

```
PS C:\Users\orian\Desktop\project1> python client5.py
Choose your nickname: Will
Welcome Will! Type '/chat <name>' to start.
```

```
/chat Fibi
Connected to Fibi!
```

```
hello Fibi
Fibi: Hey
```

```
█
```

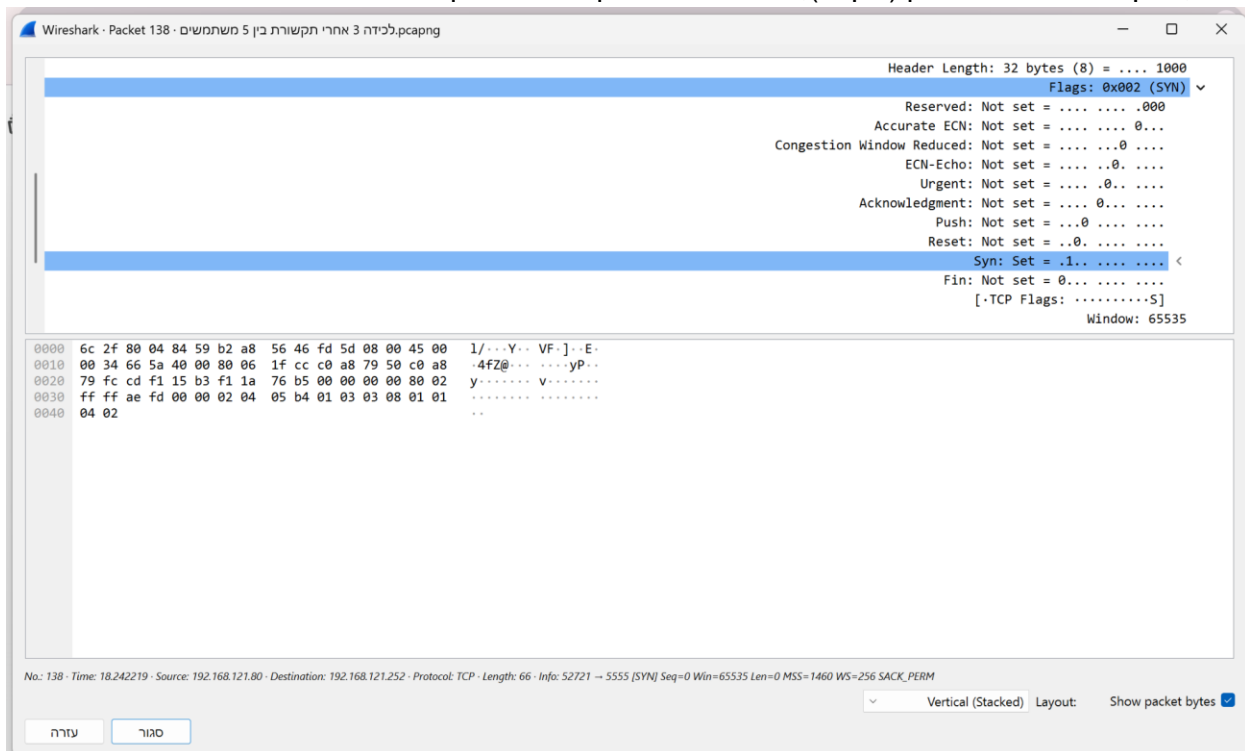
ניתוח התעבורה

לחיצת ידיים משולשת:

כאן ניתן לראות את שלושת השורות של לחיצת הידיים המשולשת - SYN, SYN-ACK, ACK. למשל בשורה הראשונה - ניתן לראות שכתובת המקור (הלקוח) היא 192.168.121.80 וכתובת היעד (השרת) היא 192.168.121.252:

No.	Time	Source	Destination	Protocol	Length	Info
138	18.242219	192.168.121.80	192.168.121.252	TCP	66	Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM [SYN] 5555 → 52721
139	18.242436	192.168.121.252	192.168.121.80	TCP	66	Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM [SYN, ACK] 52721 → 5555
146	18.666421	192.168.121.80	192.168.121.252	TCP	54	Seq=1 Ack=1 Win=65280 Len=0 [ACK] 5555 → 52721

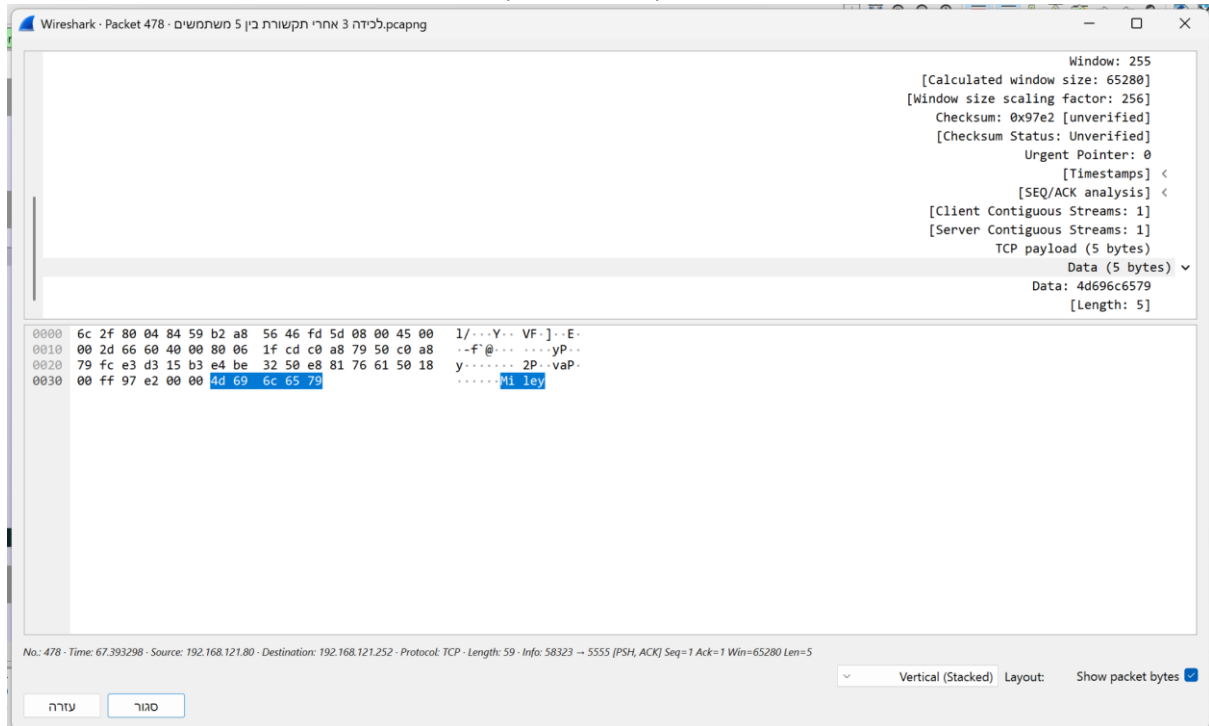
בצילום זה ניתן לראות את המבנה הפנימי של פקטת הSYN הראשונה. תחת שדה הFlags בפרוטוקול TCP, ניתן לראות שהביט דולק (ערך 1), מה שמעיד על בקשה ליצירת קשר ראשוני:



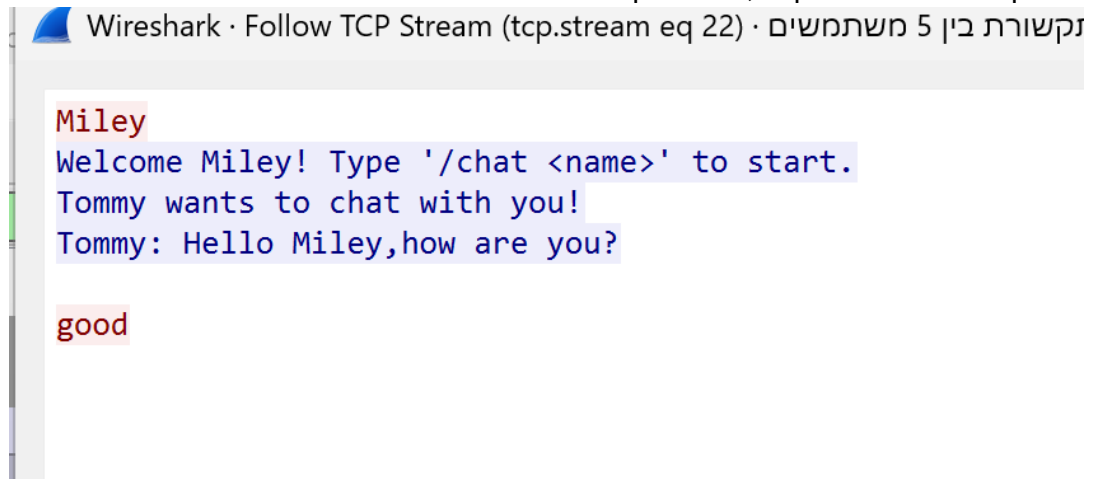
בשלב הבא מתוארת שורה של העברת מידע - כאן רואים את פקטת הPSH המכילה את המידע:

Seq=1 Ack=1 Win=65280 Len=5 [PSH, ACK] 5555 → 58323	59	TCP	192.168.121.252	192.168.121.80	67.393298	478
---	----	-----	-----------------	----------------	-----------	-----

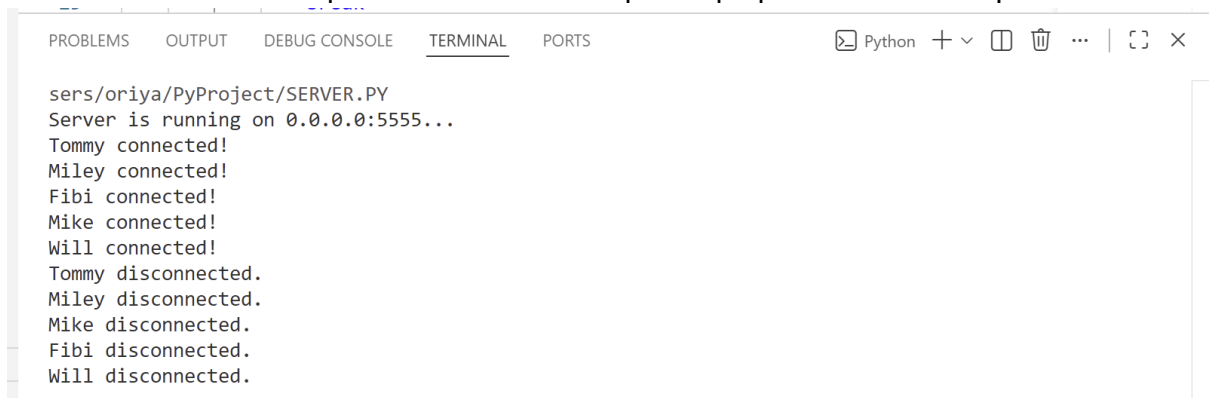
כאן ניתן לראות מידע נוסף על הפקטה שהועברה (אורכה הוא 5):



בצילום זה השתמשנו באפשרות Follow TCP Stream - כלי המאפשר את שחזור רצף הנתונים כפי שהתקבל בשכבת האפליקציה, ללא חלוקה לחבילות בודדות:



בתמונה הבאה ניתן לראות דוגמה לניתוק הקשר בעקבות סגירה יזומה מצד הלקוח:



בצילום זה ניתן לראות את תהליך סיום ההתקשרות. השורה המסומנת מציגה פקטה מסוג [RST, ACK], המעידה על כך שהלקוח ביצע ניתוק מידי והשרת אישר זאת (צילום זה נלקח מהרצה נוספת שבוצעה לצורך הדגמת תהליך הניתוק):

No.	Time	Source	Destination	Protocol	Length	Info
200	12.974894	192.168.121.80	192.168.121.252	TCP	66	Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM [SYN] 5555 → 52406
201	12.975103	192.168.121.252	192.168.121.80	TCP	66	Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM [SYN, ACK] 52406 → 5555
207	12.984451	192.168.121.80	192.168.121.252	TCP	54	Seq=1 Ack=1 Win=65280 Len=0 [ACK] 5555 → 52406
208	12.984451	192.168.121.80	192.168.121.252	TCP	59	Seq=1 Ack=1 Win=65280 Len=5 [PSH, ACK] 5555 → 52406
214	12.986009	192.168.121.80	192.168.121.252	TCP	99	Seq=1 Ack=6 Win=65280 Len=45 [PSH, ACK] 52406 → 5555
238	13.036619	192.168.121.80	192.168.121.252	TCP	54	Seq=6 Ack=46 Win=65280 Len=0 [ACK] 5555 → 52406
647	97.042837	192.168.121.80	192.168.121.252	TCP	66	Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM [SYN] 5555 → 64067
648	97.042897	192.168.121.80	192.168.121.252	TCP	66	Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM [SYN, ACK] 64067 → 5555
649	97.049726	192.168.121.80	192.168.121.252	TCP	54	Seq=1 Ack=1 Win=65280 Len=0 [ACK] 5555 → 64067
650	97.049726	192.168.121.80	192.168.121.252	TCP	59	Seq=1 Ack=1 Win=65280 Len=5 [PSH, ACK] 5555 → 64067
651	97.051890	192.168.121.80	192.168.121.252	TCP	99	Seq=1 Ack=6 Win=65280 Len=45 [PSH, ACK] 64067 → 5555
652	97.110389	192.168.121.80	192.168.121.252	TCP	54	Seq=6 Ack=46 Win=65280 Len=0 [ACK] 5555 → 64067
979	110.342281	192.168.121.80	192.168.121.252	TCP	54	Seq=6 Ack=46 Win=0 Len=0 [RST, ACK] 5555 → 64067
1049	112.710718	192.168.121.80	192.168.121.252	TCP	54	Seq=6 Ack=46 Win=0 Len=0 [RST, ACK] 5555 → 52406

בצילום זה נראים הדגלים של פקטת הניתוק. ניתן לראות שהדגל RST דולק (ערך 1). דגל זה נשלח כאשר חיבור נסגר באופן מיידי או כאשר מתרחשת שגיאה בפרוטוקול:

Wireshark - Packet 979 - Wi-Fi

Header Length: 20 bytes (5) = ... 0101

Flags: 0x014 (RST, ACK)

Reserved: Not set =000

Accurate ECN: Not set =0...

Congestion Window Reduced: Not set =0...

ECN-Echo: Not set =0...

Urgent: Not set =0...

Acknowledgment: Set = ... 1... ..

Push: Not set = ...0

Reset: Set = ...1.

Syn: Not set = ...0.

Fin: Not set = ...0.

[...TCP Flags:A..R]

Window: 0

0000 6c 2f 80 04 84 59 b2 a8 56 46 fd 5d 08 00 45 00 1/...Y... VF...E-

0010 00 28 f7 8a 40 00 80 06 8e a7 c0 a8 79 50 c0 a8 .(-@... ..yP...

0020 79 fc fa 43 15 b3 70 b8 45 af 76 c7 81 52 50 14 y-C-p- E-v-RP-

0030 00 00 7c ba 00 00 ..|...

No.: 979 - Time: 110.342281 - Source: 192.168.121.80 - Destination: 192.168.121.252 - Protocol: TCP - Length: 54 - Info: 64067 → 5555 [RST, ACK] Seq=6 Ack=46 Win=0 Len=0

Vertical (Stacked) Layout: Show packet bytes

עזרה סגור

שימוש בבינה מלאכותית

מטרות השימוש:

- השתמשנו בבינה מלאכותית במהלך הכנת הפרויקט למספר מטרות:
- כתיבת סקריפט פייתון שיצר את קובץ הCSV הראשוני עם הודעות המדמות תעבורת רשת, בהתאם למבנה הנדרש.
- דיבוג ופתרון תקלות בקוד - סיוע באיתור שגיאות לוגיות בקוד השרת והלקוח, כגון טיפול בניתוקים פתאומיים וסנכרון בין תהליכים.
- ניתוח תעבורת רשת - קבלת מידע על מבנה פקטות TCP ועל שימוש והוצאת נתונים מהWireshark, משמעות הFlags השונים וכו'.

דוגמאות לפרומפטים שנשלחו:

- "כתוב סקריפט Python שיוצר קובץ CSV המדמה הודעות HTTP בהתאם למבנה ולשדות שמופיעים בקובץ הדוגמה של המרצה. שם הקובץ צריך להיות group03_http_input.csv ועליו להכיל 130 שורות לפחות ולכלול את העמודות הבאות: msg_id, app_protocol, src_app, dst_app, src_port, dst_port, message, timestamp."
- "איך ניתן להתאים את מחברת הג'ופיטר הנתונה כך שתטען את קובץ ה-CSV שלנו?"
- "איך ניתן לראות את כל ההודעה שהועברה בצ'אט בבת אחת במקום לראות רק פקטות נפרדות?"
- "למה הלקוח מתנתק כשפותחים חלון נוסף ב-VS Code?"